OS

UNIT – 1

OS → critical software layer that manages comp hardware & software, giving stable & consistent environment for applications to run.

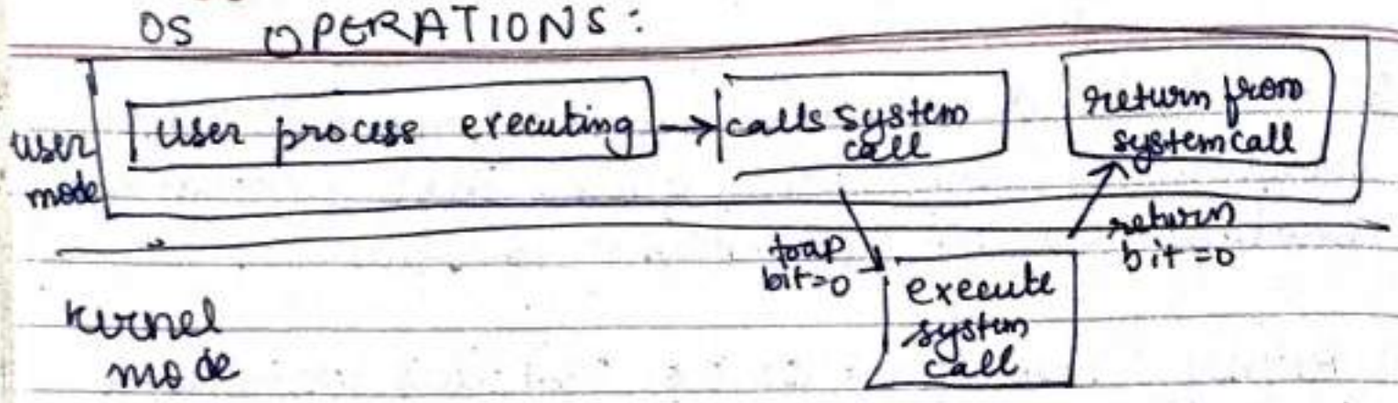
Kernel : core part of OS that acts as interface b/w hardware & user application

User Thread	Kernel Thread
→ managed by user level libraries without kernel supp	→ managed directly by Operating system kernel.
characteristics: → Managed by user ~~level lib~~ space	→ Kernel space
→ Fast creation and switching	→ Slower in switching
→ One thread block → entire process blocks	→ One thread blocks → others can run.
Adv: → Fast switching → low overhead	→ Better performance on multiprocessor
Dis: → cannot utilize multiprocessor systems efficiently → Blocking	→ Slower switching → more overhead.
Ex: POSIX threads, Java green	Ex: Linux threads, Windows threads

User view : refers to how OS appears to user as an interface that make computer easy to use.

System view : how OS manages hardware resources efficiently

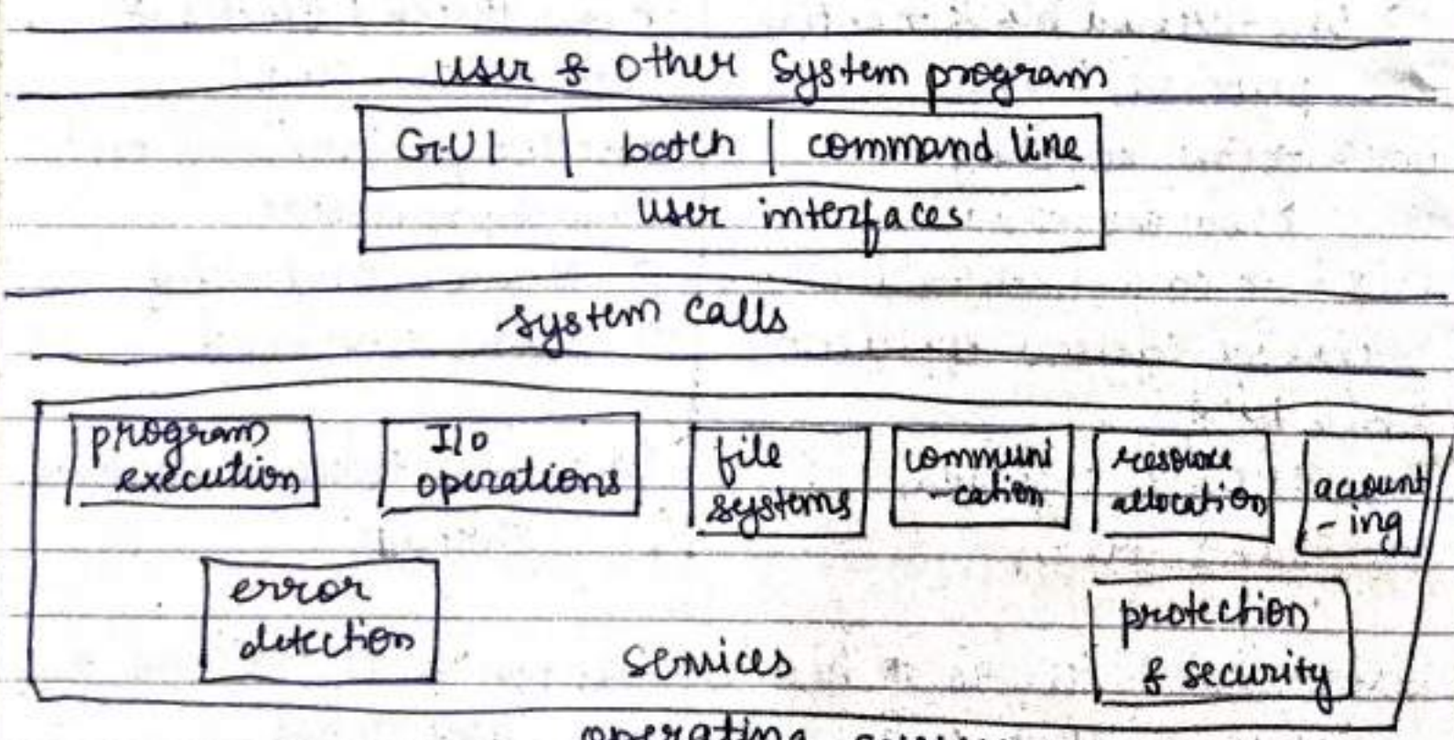
OS OPERATIONS:



User mode : User application run with limited access to hardware & system resource

Kernel mode : OS runs with full access to hardware & system resource.

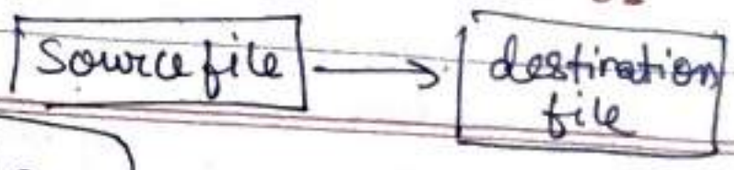
OPERATING SYSTEM SERVICES

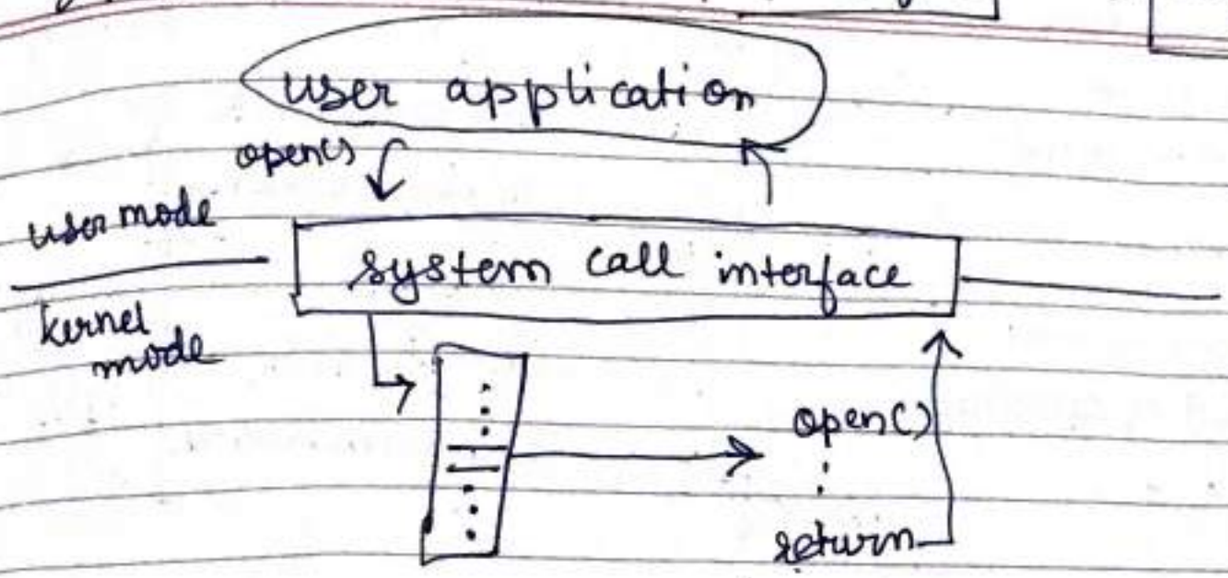


300
270

270
60
60

SYSTEM CALL





TYPES OF SYSTEM CALL

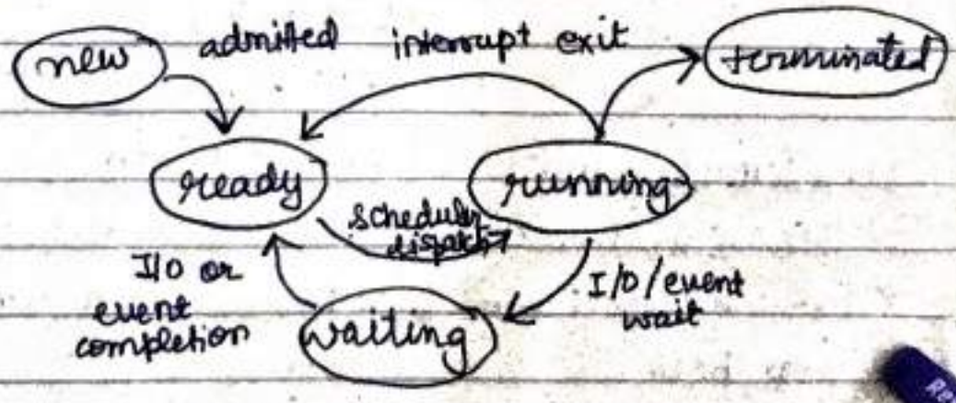
PDF PIC

i) Process control : hault execution normally/abnormally
ii) Device Management :
iii) File management :
iv) Protect : controlling access to resource
v) Information Maintenance - return current time() & date()
vi) communication : message passing model & shared memo model

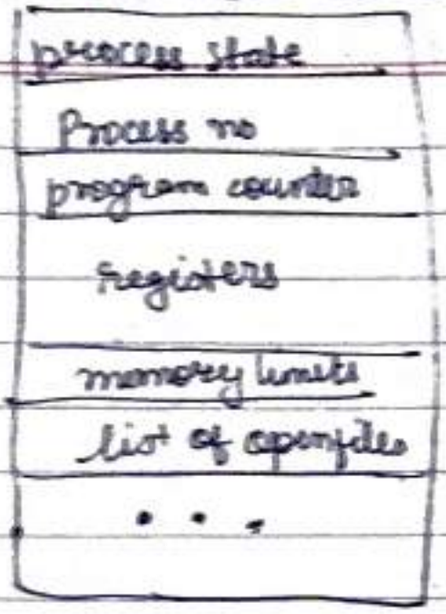
PROCESS MANAGEMENT :

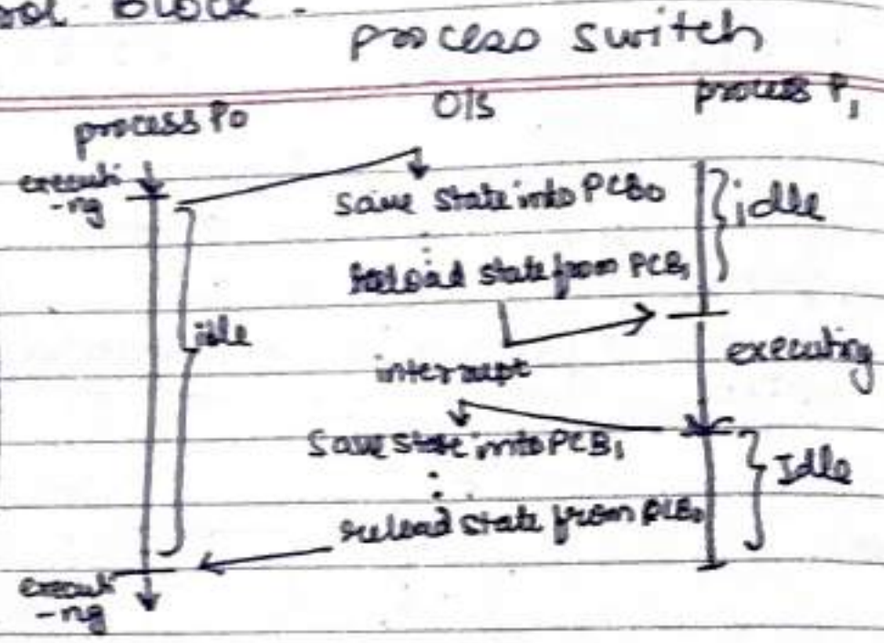
Process state :

1. New.
2. Running
3. Waiting
4. Ready
5. Terminated

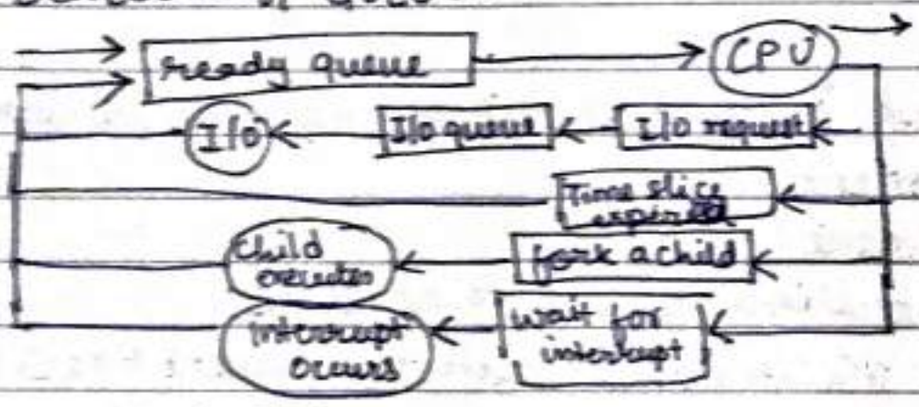


82 Process control Block

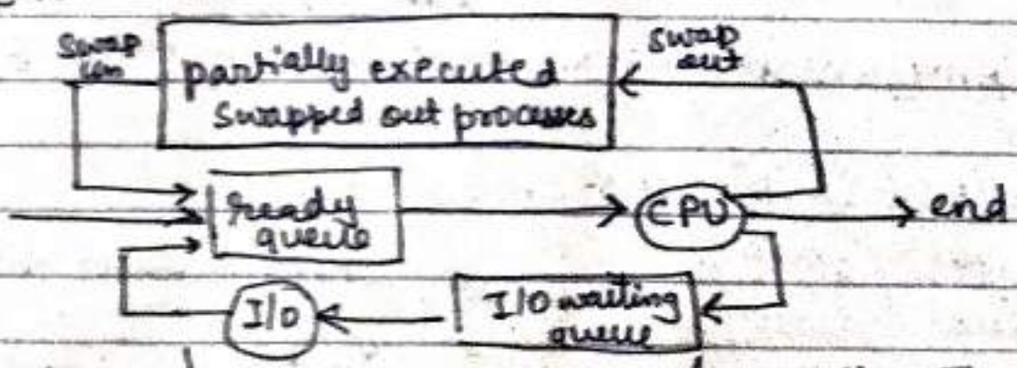




SCHEDULING QUEUES



SCHEDULERS



Long Term Scheduler	Short Term	Medium Term
→ Job Scheduler	→ CPU Scheduler	→ Process swapping scheduler
→ Takes process from job pool	→ Process from Ready state	→ Process from running or dead state
→ Speed less than short term scheduler	→ Fastest among all	→ B/w two others
→ controls degree of multiprogramming	→ less control	→ reduces degree.

CPU scheduler	Job scheduler
→ ~~Allocates CPU~~	
→ Selects process from ready queue to execute	→ Selects job from job pool to load into memory
→ Also called short term scheduler	→ Long term schedulers.
→ Selects from ready ~~queue~~ queue	→ Selects from job pool.
→ Running state → moves to	→ moves to ready state
→ Very fast speed	→ slow
→ Runs very frequently	→ Runs less frequently.
Ex. Round Robin.	Ex. First come First serve.

I/O bound Process	CPU bound Process
→ Spends time performing I/O ~~pro~~ operations	→ Performing CPU operations
→ Relies heavily on I/O devices	→ CPU for processing tasks
→ Waits for I/O operation to complete tasks → short CPU burst	→ long CPU burst.
→ Ex. Reading/writing to disk	→ Complex calculations.
→ CPU idle while waiting for I/O operations	→ CPU busy

CONTEXT SWITCHING : → Adv efficient CPU utilization.
→ Multitasking → Responsiveness → Fairness.
Dis → Overhead → Complexity → Cache misses

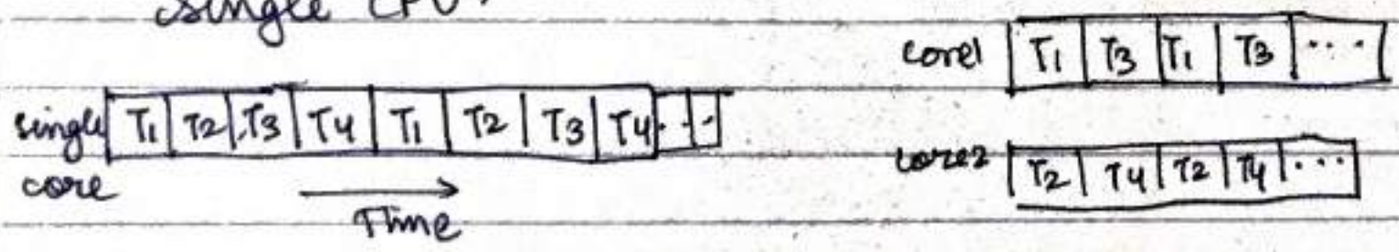
UNIT-2

Multithreading : ability of process to execute multiple threads concurrently within same process

Benefits:

1. Responsiveness : allows program to remain responsive even when part of it is performing a long task. If one thread is busy, other threads continue execution.
2. Resource sharing : Resources such as memory, files & data within same process
   Threads share code, data, files.
3. Improved performance : Multithreading improves CPU utilization & system performance. Multiple threads execute simultaneously reducing idle CPU time.
4. Scalability / Better utilization of multicore system: Multithreading allows parallel execution on multiple CPU cores. each thread run on separate core.
5. Economy : creating threads is faster & uses fewer resources than creating processes. Threads share memory, so overhead is less.

Multi core programming : Process of writing software that can run on systems with multiple cores in single CPU.



Key concepts : 1. concurrency & Parallelism.
2. Multithreading 3. Synchronization
4. Data sharing & memory access 5. Scalability
6. Balancing load 7. Thread safety

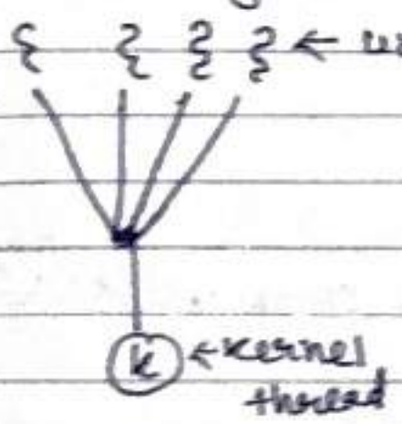
Challenges: 1. Identifying task 2. balance
3. Data splitting 4. Data dependency 5. Testing & Debugging

PARALLELISM: executing multiple tasks simultaneously on different cores.

1. Text based parallelism: Dividing program into independent tasks that can run concurrently on different cores.
2. Data parallelism: Dividing data across multiple cores & performing same operation on each partition of data set in parallel.

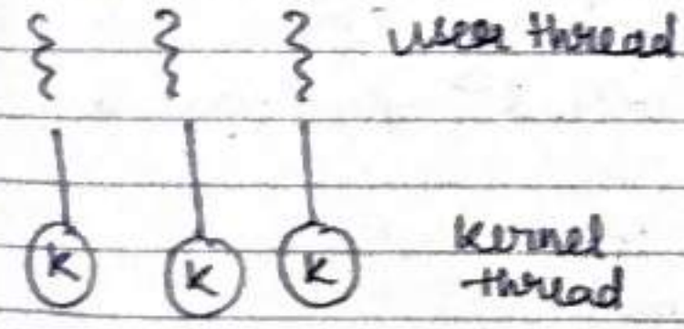
MULTITHREADING MODELS:

1. many to one



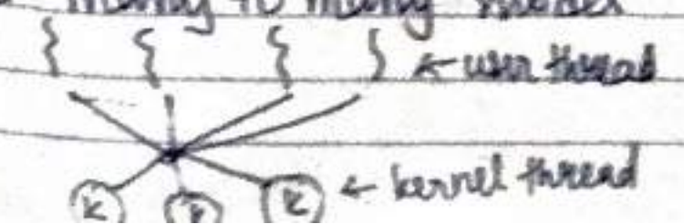
Efficient
Entire process block if a thread makes blocking system call.
One thread access kernel at a time, multiple thread are unable to run in parallel on multicore systems. Ex. Green Thread

2. One to One model



Allows another thread to run when thread makes blocking system call.
Creating so many thread can burden performance of application.
Ex. Linux

3. Many to many model



CPU Scheduling & Process synchronization

i> CPU - I/O Burst cycle

load store } CPU burst
wait for I/O } I/O burst

ii> CPU scheduler : CPU selects process in ready ~~queue~~ queue to execute it & allocates to CPU.

iii> Preemptive Scheduling:
1. running to waiting state
2. Running state to ready state (Interrupt occurs)
3. Waiting to ready state
4. process terminates

Dispatcher : module that gives control of CPU to process selected by short term schedulers.

SCHEDULING CRITERIA:
1. CPU utilization:
2. Throughput : no of processes completed per ~~hour~~ time unit
3. Turnaround time
4. Waiting time 5. Respond time.

Turnaround Time = Completion Time - Arrival Time / Burst Time
+ Waiting Time

Waiting Time = Turn around Time - Burst Time

FCFS → Dis = conveyor effect
SJF → Dis → starvation
RR →
Priority Scheduling → Dis → Starvation

AMDAHL'S LAW: theoretical maximum speedup of program when part of it is parallelized using multiple processors/threads.

$$S = \frac{1}{(1-P)+\frac{P}{N}}$$

DIFFERENT KERNEL ARCHITECTURE:

1. Monolithic kernel: All OS services, such as process management, memory management & device driver run in the kernel space.

Adv: → High performance due to direct communication
→ Simpler design & implementation.

Dis: → Single failure, crash entire system.
→ Difficult to maintain

Ex. Linux.

2. Micro kernel: minimizes functionality running in kernel space.

Adv: → Failure in user space service doesn't crash kernel
→ Easier to extend & maintain

Dis → Slower performance
→ complex design

Ex Mach

3. Hybrid kernel: combines features of monolithic & micro kernels.

Adv: → Better performance
→ Enhanced flexibility

Dis: → Increased complexity in design & maintenance
→ Larger resource usage

Ex. mac OS.

4. Exokernel : ~~provides minimal abs~~ allows applications to directly manage hardware resources.

Adv : → Highly flexible
→ High performance

Dis : → complex to develop & debug
→ limited support & adoption

Ex. ExOS

5. Nanokernel: provides only most essential hardware, leaving all system service implemented externally.

Adv → Extremely small & efficient
→ High modularity

Dis → Limited functionality
→ complex to develop & maintain

Functions of kernel — PDFPMC

1. Process management : (4) Memory man
2. Device Management (5) Communication
3. File system " (6) Protection

UNIT-3

FILE OPERATIONS: (system call)

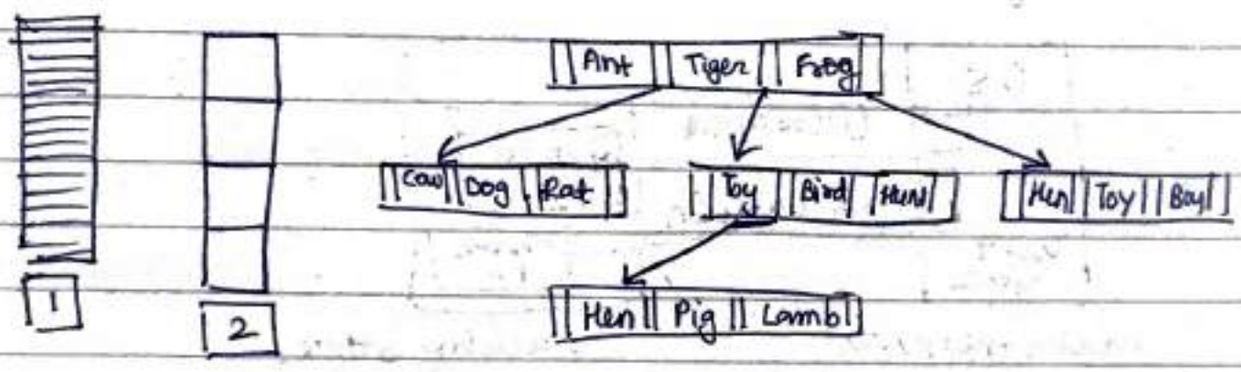
1. Creating file 2. Writing a file
3. Reading file 4. Repositioning file
5. Deleting file

FILE STRUCTURE – 3 ways

[1] File Structure 1: File is unstructured sequence of byte. Therefore OS doesn't care about file, as all it sees are bytes.

[2] File Structure 2: File is a sequence of fixed length records each with some internal structure. Read operation returns record & write operation just appends record.

[3] File Structure 3: File consists of tree of records not be of same length, Tree is sorted.



FILE ACCESS METHODS:

1. Sequencial access: Records are accessed in some sequence. Info in file is processed on after the other. Ex. compilers access file in fashion.

Adv: → simple program & easy design
→ best use if storage space

Dis: → Time consuming
→ High data redundancy
→ Random search not possible

2. Direct Access: Files are directly & random accessed. Ex. Databases

Adv: → Helps in online transaction processing system like railway reserve
→ updates several files quickly.

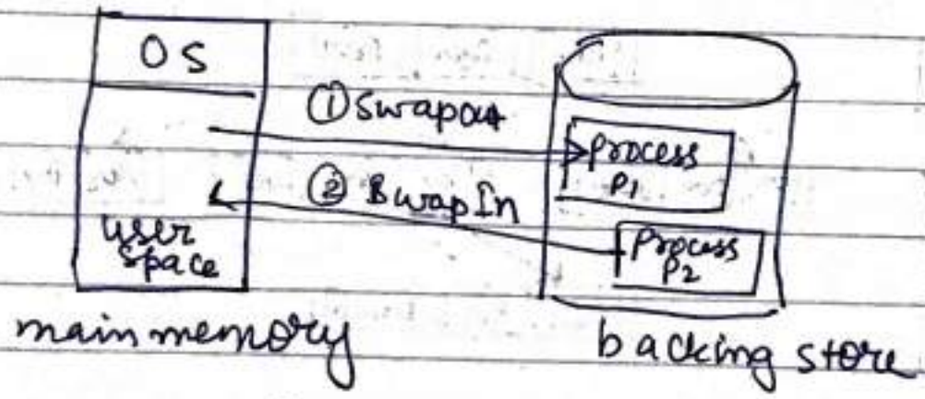
Dis → Doesn't provide backup
→ Expensive

3. Indexed Sequential Access: First access file directly then access sequencially.

Adv: → Access record very fast
→ Records can be inserted in middle of file

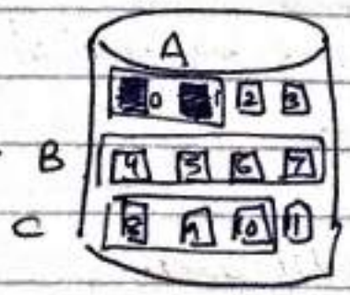
Dis → requires unique key
→ takes longer time to search index of data
→ Expensive.

SWAPPING: Process can be swapped temporarily out of main memory → secondary storage to make memory available to other processes.



FILE SPACE ALLOCATION

1. Contiguous allocation: file occupies contiguous set of blocks. Ex. b, b+1, b+2 . . . , b+n-1.



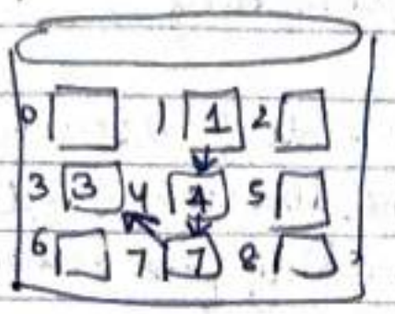
Directory

	start	length
A	0	2
B	4	4
C	8	3

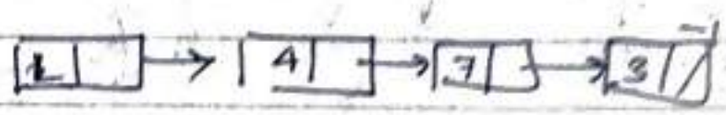
Adv 1) Easy to implement
2) excellent read performance

Dis: 1) Difficult to grow file
2) Disk becomes fragmented.

2. Linked list allocation : Each file is linked list of disk block.



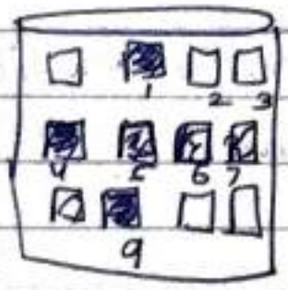
Directory	Directory
~~1 — 3~~ end	File Start
	A 2



Adv : → no external fragmentation
→ File size can be changed.

Dis : → Large seek time
→ overhead of Pointers.

3. Indexed file allocation.



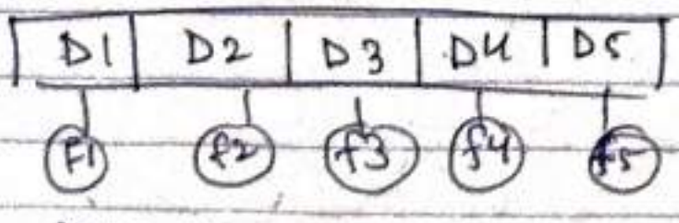
File A — Index block 6 → 9, 1, 4

Adv : → Support ~~extern~~ direct access
→ No external fragmentation

Dis : → Pointer overhead
→ Multilevel index

DIRECTORY : Container that is used to contain folders & files.

1. Single level directory:

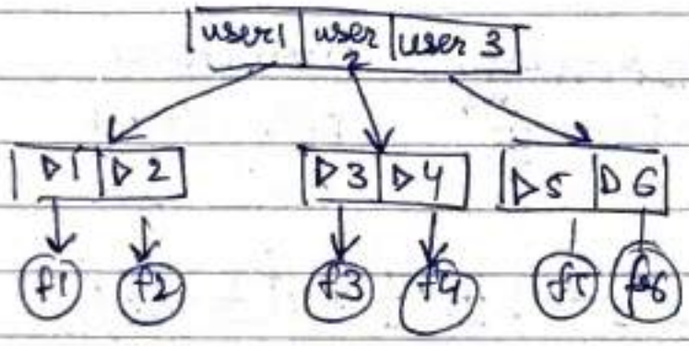


Adv : → simple, implementation is very easy.
→ ~~Small~~ files smaller in size, searching is faster

Dis → 2 files can't have same name.
→ Cannot group same type of file together.

2. Two level directory

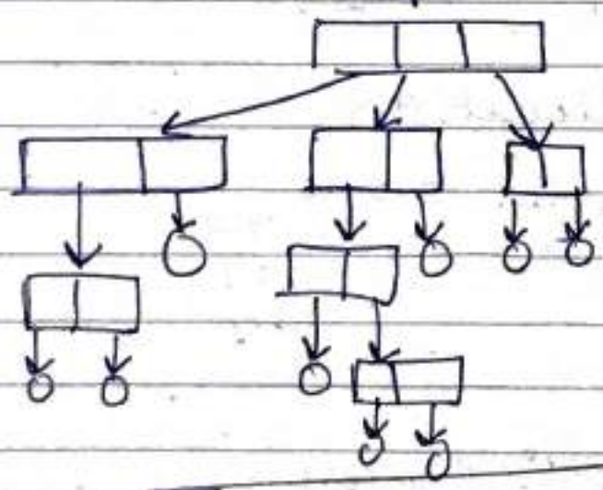
each user has own user file directory.



Adv :→ diff user name same directory & file name.
→ Searching file → easy

Dis : → user can't share file with other user.
→ Not scalable.

3. Tree level directory : most common, every file has unique path

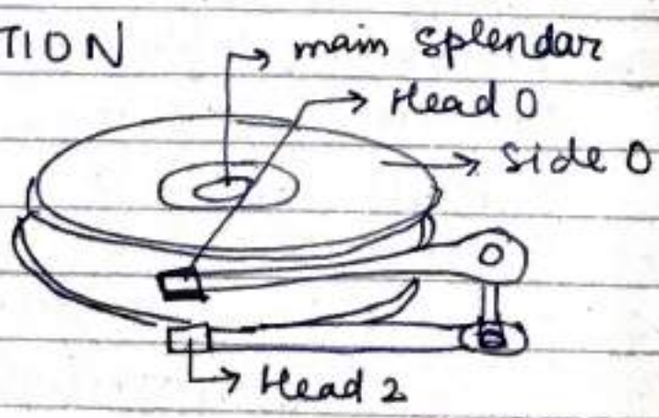


Adv → Scalable
→ Searching very easy

Dis :→ Cannot share file
→ Inefficient

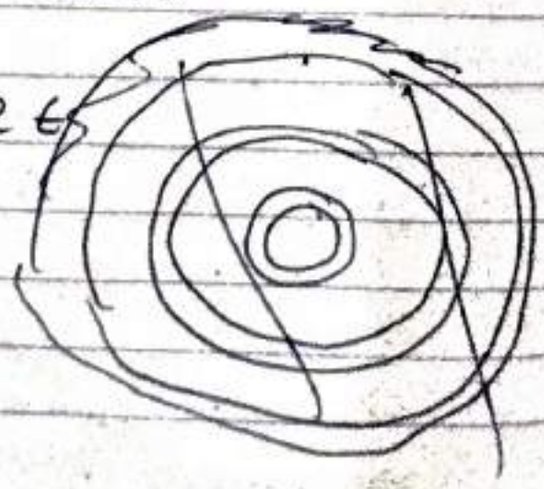
DISK ORGANISATION

1. Seek Time :
2. Rotational latency
3. Data transfer time
4. Controller time
5. Avg access time

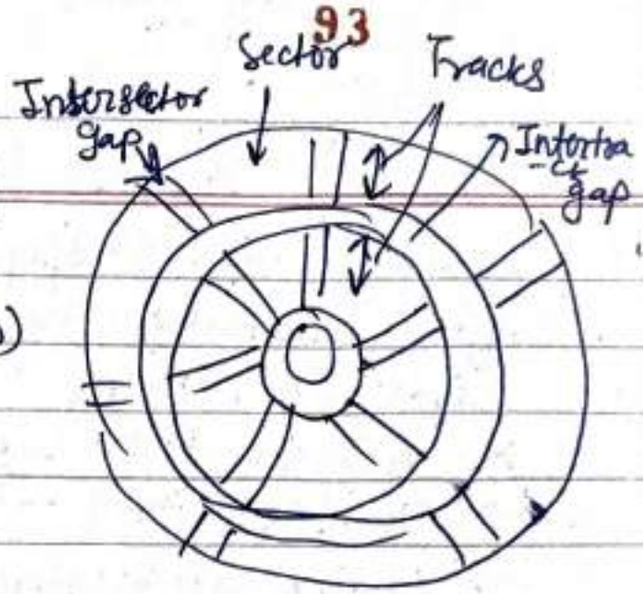


LOGICAL DISK STRUCTURE

disk → ~~tracks~~ tracks → sectors

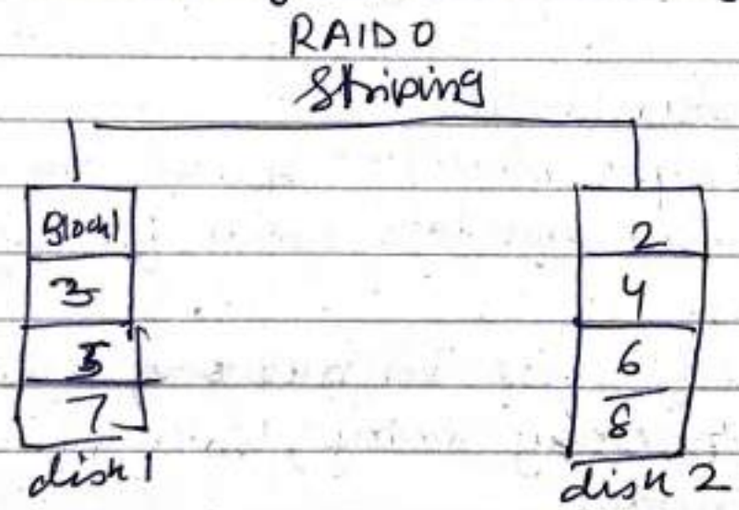


1. MBR (Master Boot Record)
2. DBR (DOS Boot Record)
3. FAT (File allocation Tables)
4. Root Directory
5. Data Area.



RAID levels:

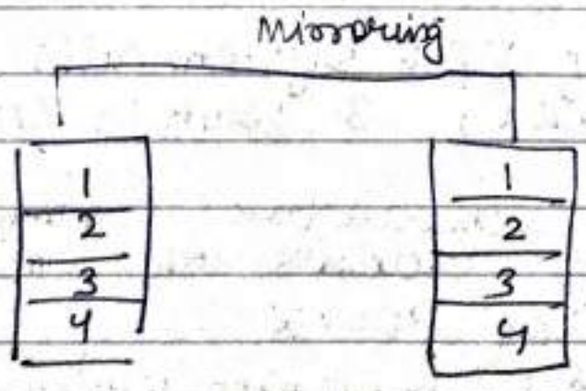
RAID 0 → striping, no redundancy of data



→ Best performance

→ No fault tolerance

RAID-1 disk mirroring



RAID-2

OS Allows multiple types of file systems to be integrated into directory sg system.

VFS : kernel level abstraction layer that enables OS to support multiple, diverse file systems through single, unified interface. Roles:

→ Abstraction layer:
→ Unified API –
→ Uniform system call
→ Support diverse file systems

VFS data structures

1. Superblock (super-block) – Specific mounted file system, containing metadata about file system type, size & status.
2. Inode : Specific file in memory
3. Dentry : directory entry, linking file names to inode.
4. File : Open file associated with process

Implementation of VFS system calls

1. Open() 2. Read() 3. Write() 4. Close()

File locking : → Shared lock → Advisory
Exclusive lock → mandatory

1. Advisory locks : by POSIX
2. mandatory locks – by System V release 3
3. Linux supports both

File System → MS DOS
→ UNIX V7

FILE TYPES:	PATHNAME LOOKUP
disc based	converts textual path into inode reference
Network based	
memory based	
Pseudo filesystem	

System architecture

file system interface
↓
VFS interface

- local file system type1 → disk
- local file system type2 → disk
- remote file system type 1 → network

1. Inode object: represents individual file
2. file object: represents a open file
3. superblock object: represents entire file system
4. dentry object: represents an individual directory entry

UNIT-3

CRITICAL SECTION PROB:

```
do{
    entry section
    critical sec
    exit sec
    remainder sec
} while (true);
```

satisfy 3 requirements:

1. Mutual exclusion: No other process can be executing in critical sec
2. progress:
   No process is executing, then process not executing in remainder section can participate in deciding which will enter its critical section next.
3. Bounded wait: limit on no. of times other processes allowed to enter critical section after process had made request to enter its critical section.

HANDLE CRITICAL SECTION:

1. Preemptive kernel: Allows process to be preempted while running in kernel mode.
2. Non preemptive kernel: Kernel mode process will run until it exits kernel mode & blocks control of CPU.

RACE CONDITION : Two or more process / thread access shared resources simultaneously, & final outcome depends on specific order of execution.

Prevent it : 1. Locks/Mutexes : only 1 thread access shared resource at a time.

2. Semaphores : Limit access to shared resources – controlling no. of threads that can use resources simultaneously.

3. Monitors : High level constructs manage access to shared resources in controlled manner.

CRITICAL SEC → SOLUTION:

1. PETERSON'S ~~ALGO~~ SOLUTION : software based solution → uses 2 variables (flags) to achieve mutual exclusion & ~~progress~~ prevent race conditions → 2 / more processes need to access shared resource.

```
do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);
        critical section
    flag[i] = false;
        remainder section
} while true;
```

process i wants to enter critical sec

turn to process j

keeps process i waiting if flag[j] = true & turn == j.

→ i enters critical sec & perform operation on shared resource

→ i exists critical sec, & does no want to enter critical sec

ADV : simple & efficient

DIS : works only for 2 process

test-and-set

```
int compare-and-swap (int * value, int expected, int new-value){
        int temp = * value;
        if (*value == expected)
                * value = new-value;
        return temp;
}
```



```
do {while (compare-and-swap(&lock, 0, 1) != 0)
        ;
        [critical section]
        lock = 0;
        [remainder sec]
} while (true);
```

2. MUTEX LOCK : prevent race condition when multiple processes access shared resources.
Only 1 process can access critical section at a time.

```
lock available:  acquire(){
                        while (!available)
                                ;
                        available = false;;
                }

release (){
        available = true;
}
```

```
do{ [acquire lock]
    critical section
    [release lock]
    remainder sec
} while (true);
```

spinlock

DIS : → issue of busy waiting : other processes have to rapidly check if lock is available or not → wasting CPU cycle → poor performance

ADV : → No need context switch

SEMAPHORES: manage access to shared resources by multiple process avoid race conditions/deadlocks

1. Counting semaphores: multiple processes to access a resource till certain limit.
2. Binary semaphores: allowing 1 process to access critical section at a time.

```
wait(S){
        while (S<=0)
            ;   # busy
        S--;
    }
signal(S){
        S++;
    }
```

Semaphore	Mutex
→ have value greater than 1	→ Binary value
→ multiple instances of resource	→ single access resource
→ complex	→ Simple Binar lock.
→ value =0, thread will have to wait until another thread releases it.	→ Locked, thread trying to lock it will block, unit mutex released.

DIS OF SEMAPHORES :

1. Deadlocks : 2 or more processes are waiting indefinetly for events that can be triggered by one of those processes.

2. Starvation : process waits indefinitely to acquire semaphore.

PRIORITY INVERSION : higher priority waits for lower priority to release resource.

Solution Priority Inheritance Protocol :

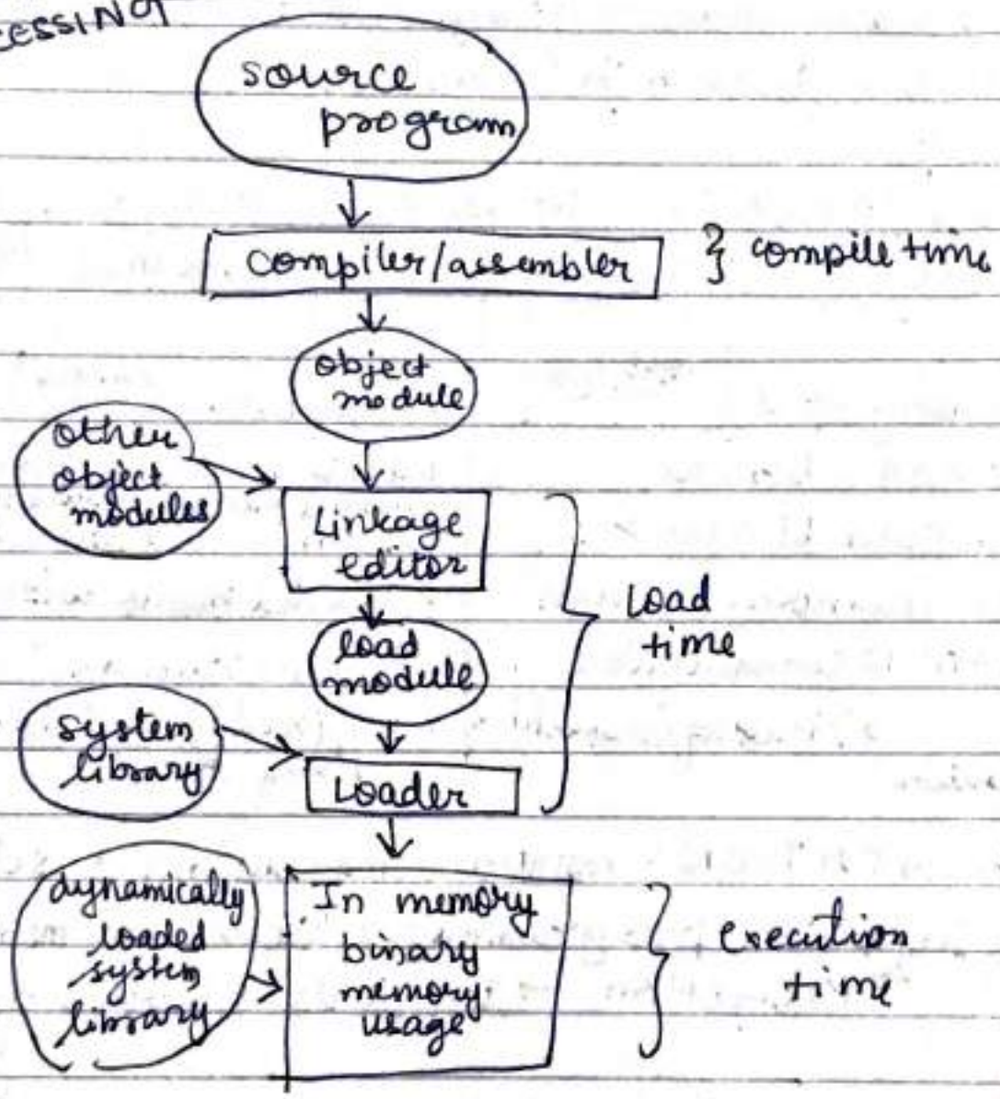
1. Producer - Consumer → semaphores, mutex lock.
2. Reader-Writer → Semaphores, mutex
3. Dining Philosopher → Prob: mutual exclusion, deadlock, starvation, Concurrency & synchronization.

99

Scheduling & Process: $U_{max} = n(2^{\frac{1}{n}} - 1)$

UNIT-4

MULTISTEP PROCESSING



Swapping → 1> Standard swapping 2> Modified

contiguous Memory allocation:
memory Protection: 1. Relocation register
2. Limit register

memory allocation strategies:
1. Fixed Partition: Fixed size partition, simple & flexible
2. Variable partition: Process gets exactly memory it needs

1. FIRST FIT : Adv : → Simple, fast → minimal search time
Dis → small unusable fragments at beginning of memory. → Does not alway utilize memory optimally.

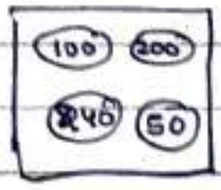
100

2. BEST FIT : Adv → min wasted space
Dis → Slower because scan entire memory
→ Small, unusable holes.

3. WORST FIT: Adv → Larger leftover holes can be used in future
Dis → Creates larger fragmentation over time as big holes broken into smaller.

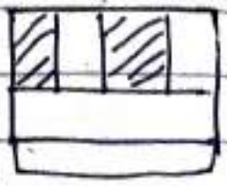
FRAGMENTATION : 1. External : free memory

External



→ Free memory split into small blocks.
→ Unused space b/w block.
→ Total memory enough
→ solution: 1) compaction
2) Paging/segmentation
→ segmentation

Internal



→ Unused memory space within allocated block waste memory
→ wasted memory, unused space within allocated block.
→ wasted memo within blocks
→ Sol → dynamic allocation.
→ fixed sized blocks.
→ Paging

SEGMENTATION : memory management scheme that supports programmers view of memory.
Adv : → Protection → Supports Sharing → efficient memory utilization
Dis : → external fragmentation → Complex management.

Paging	Segmentation
→ Logical memo into fixed sized block called pages	→ Divides logical memo into variable sized sements.
→ Fixed size	→ Variable size
→ Logical address consist of pages no & page offset	→ segment no & segment offset
→ User does not see pages	→ Visible to user
→ Protection at page level	→ segment level.
→ Ex. modern OS	Ex → Earlier system like Intel 8086

PAGING:

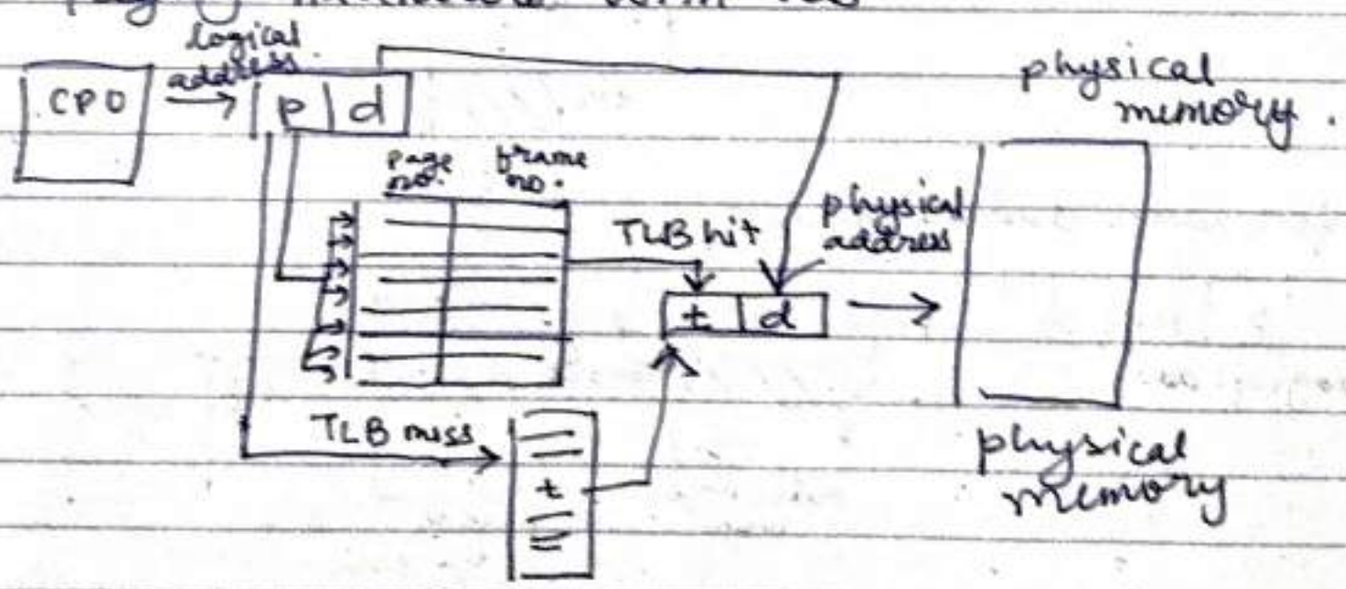
1. Frame: fixed size block of physical memory
2. Page: Fixed size block of logical memory.
3. Page Table: maps page no to frame no.
4. Backing store: divided into blocks same size as frame

Translation Lookaside Buffer: fast cache used in memo management to improve speed of virtual address translation.

Page Table Base Register: store starting address of page table for currently running process.

TLB (")	Page Table
→ Located in MMU	Stored in main memo
→ stores small no of entries	→ Store all mappings for process.

Paging hardware with TLB



Effective memory access Time:

EAT = (Hit ratio) × (Time for TLB Hit) + (miss ratio) × (Time for mm)

Page Table Length Register: indicates size of page table for a process.

Func: 1. defining Page Table Boundaries
2. Preventing invalid access 3. Dynamic process support

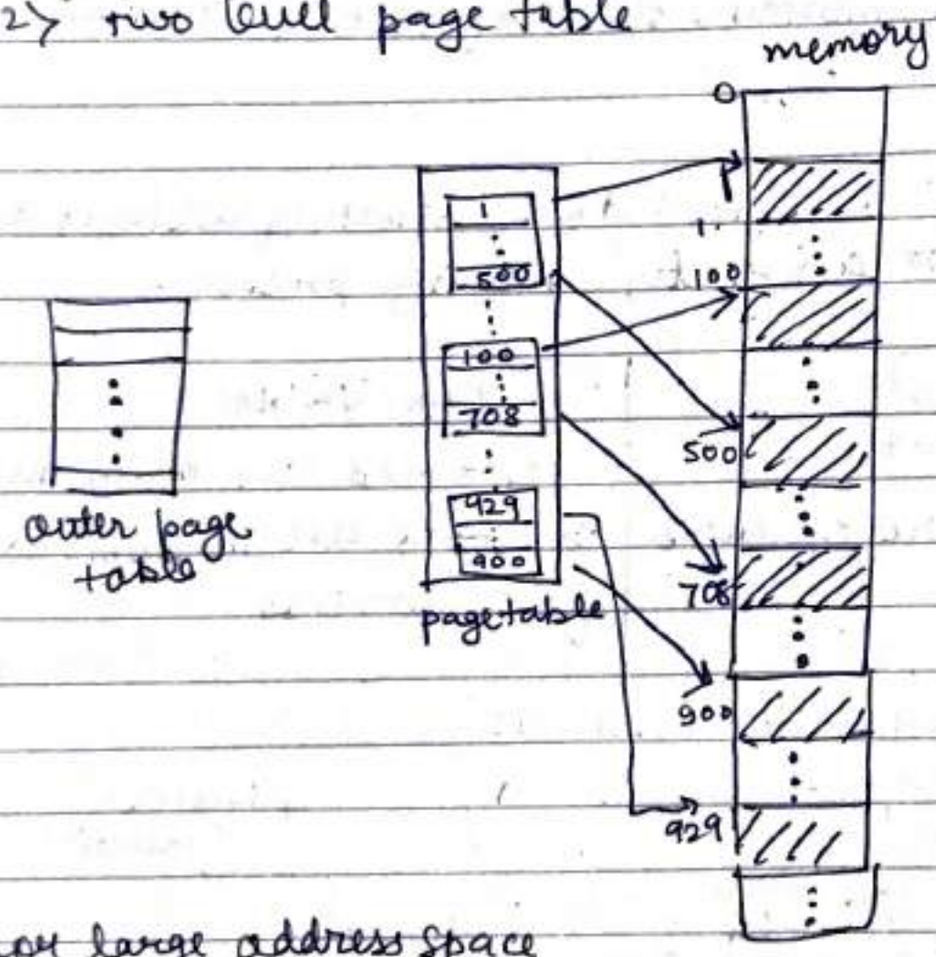
STRUCTURE OF PAGE TABLE:

i> Hierarchical Paging : logical address upto (32/64 bits)

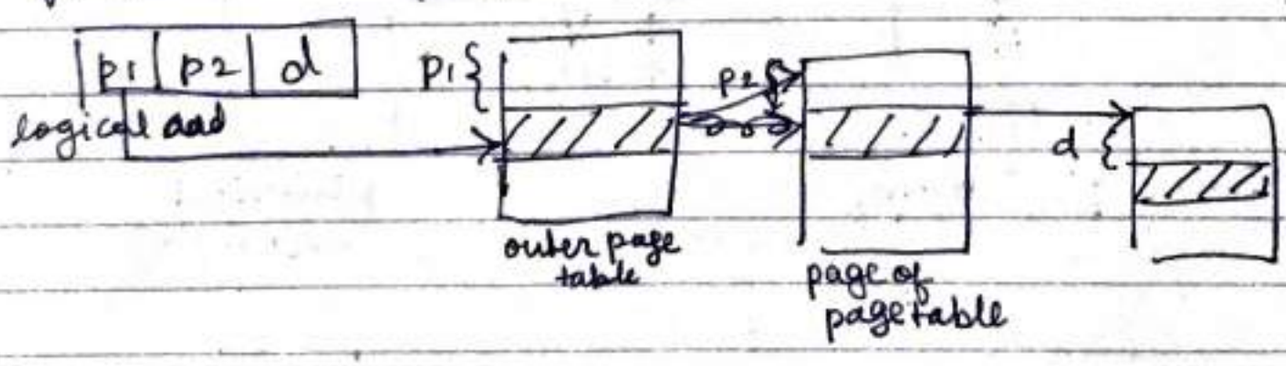
2 ways:

1> page table broken into smaller manageable pieces,

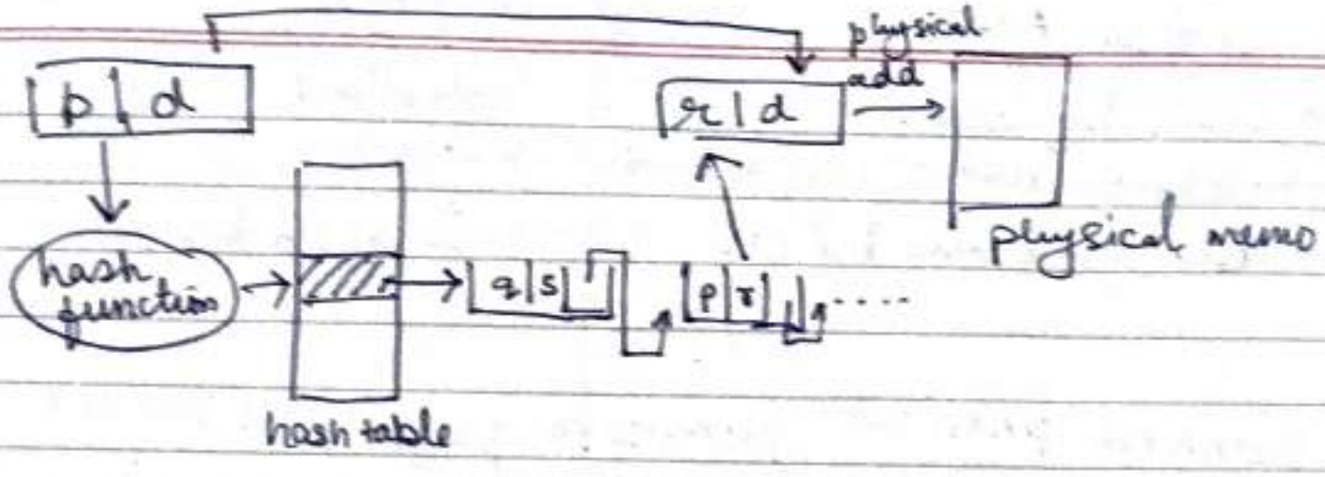
2> two level page table



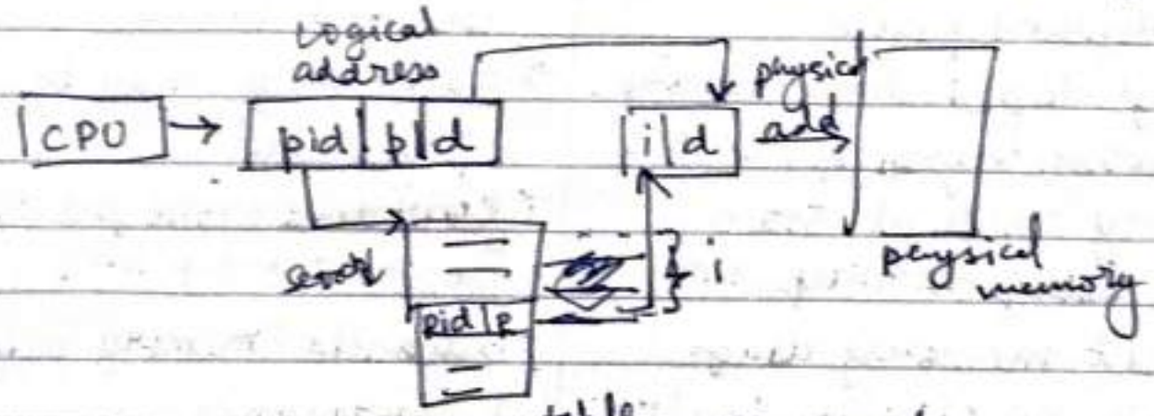
for large address space



ii> Hashed Page Table : address space larger than 32 bit linked list

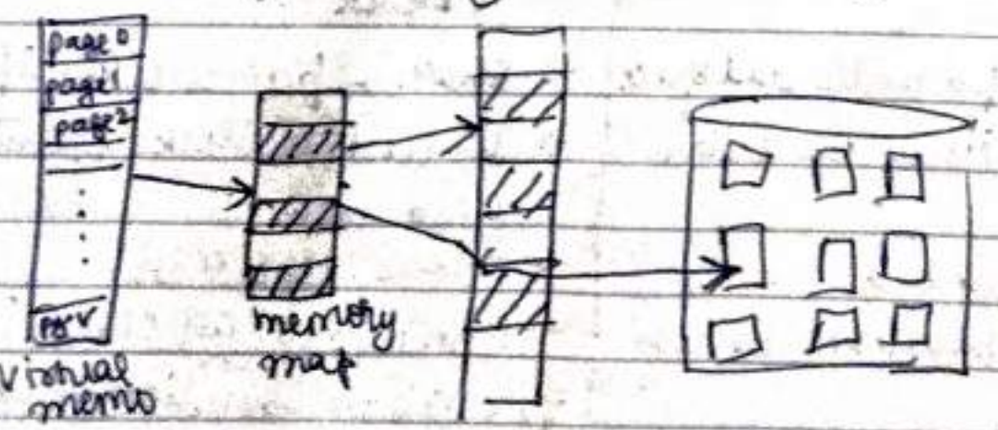


iii) Inverted Page Tables : each process has its own page table that maps virtual add to physical memo



Only 1 page table for entire system.

VIRTUAL MEMORY: separates process's logical memory from physical memory.



Virtual Address Space : range of virtual add that OS can process

1. Virtual vs Physical memory
2. Address Space Layout
3. Sparse

Logical Address	Virtual
→ Seen by user	Invisible
→ ~~determined during execution~~	→ ~~MMU~~
→ Add generated by CPU	→ managed by OS.

DEMAND PAGING: program pages are loaded into memory only when they are needed during execution.

Pure demand paging	Demand paging
→ No page loded into memory. only when refered for 1st time	→ Some page may be preloaded.
→ memory empty at start.	→ contains some preloaded page
→ Higher ~~Startup~~ startup time.	→ Faster startup time.
→ Efficient memory usage	→ Balances memory usage & performance
→ High no of page fault	→ Few page fault.
→ ~~More efficient~~	

fork	vfork
→ Duplicates parent's add space	→ shares parents add space
→ Parent continues execution after fork	→ Parent is blocked until child exits.
→ Higher overhead	→ more efficient.
→ General purpose process creation	→ Optimized for child processes immediately calling.

THRASHING: OS spends more time swapping pages → decreases CPU efficiency & utilization

Sol → 1. Local replacement 2. Working set model 3. Page fault freq

Q. Available memo: 600, 350, 200, 400, 500
Process : 120, 430, 250, 350, 100

First Fit : 120 → 600 − 120 = 480

480, 350, 200, 400, 500

430 → 480 − 430

50, 350, 200, 400, 500

250 → 350 − 250 = 100

50, 100, 200, 400, 500

350 → 400 − 350 = 50

50, 100, 200, 50, 500

100 → 100 = 0

50, 100, 200, 50, 500

Worst Fit : 120 → 600 − 120 = 480

480, 350, 200, 400, 500

430 → 500 − 430 = 70

480, 350, 200, 400, 70

250 → 480 − 250 = 230

230, 350, 200, 400, 70

350 → 400 − 350 = 50

230, 350, 200, 50, 70

100 → no space

Best Fit : 120 → 200 − 120 = 80

600, 350, 80, 400, 500

430 → 500 − 430 = 70

600, 350, 80, 400, ~~500~~ 70

250 → 350 − 250 = 100

600, 100, 80, 400, ~~500~~ 70

350 → 400 − 350 = 50

600, 100, 80, 50, ~~500~~ 70

100 → 100 − 100 = 50

600, 0, 80, 50, ~~500~~ 70.

FIFO : 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

	7	0	1	2	0	3	0	4	2	3	0	3	2
	7	7 0	7 0 1	2 0 1		2 3 1	2 3 0	4 3 0	4 2 0	4 2 3	0 2 3		
	1	2	3	4	★	5	6	7	8	9	10	★	★

	1	2	0	1	7	0	1
	0 1 3	0 1 2			7 1 2	7 0 2	7 0 1
	11	12	★	★	13	14	15

Page fault = 15

Page hit = 5

Page fault frequency $= \frac{15}{20} \times 100 = 75\%$

Page hit freq $= \frac{5}{20} \times 100 = 25\%$

LRU

	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0
	7	7 0	7 0 1	2 0 1		2 0 3		4 0 3	4 0 2	4 3 2	0 3 2			1 3 2		1 0 2
	1	2	3	4	★	5	★	6	7	8	9	★	★	10	★	11

	1	7	0	1
		1 0 7		
	★	12	★	★

Page fault = 12

Page hit = 8

LFU :

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2
7	7	7	2		2		4	4	3			1 0 3	3	3
	0	0	0		0		0	0	0				0	0
		1	1		3		3	2	2				1	2
1	2	3	4		5		6	7	8				9	10

frequency Table

0		1	1	1	2	2	3	3	3	3	4	4	4	4	4
1			1	1	1	0			0	0	0	0	0	1	0
2				1	1	1	1	0	1	1	1	1	2	0	1
3						1	1	1	0	1	1	2	2	2	2
4						0		1	1	0	0	0	0	0	0
7	1	1	1	0		0									

Page Fault = 10 $\frac{10}{15} \times 100 = 66.67\%$

Page Hit = 5 $\frac{5}{15} \times 100 = 33.33\%$

MFU

2	3	2	1	5	2	4	5	3	2	5	2
2	2		2	5	5	5		3		3	3
	3		3	3	2	2		2		5	5
			1	1	1	4		4		4	2

Frequency Table

1				1	1	1	0	0	0	0	0	0
2	1	1	2	2	0	1	1	1	1	2	0	1
3		1	1	1	1	0	0	0	1	1	1	1
4							1	1	1	1	1	0
5					1	1	1	2	0	0	1	1

Page hit = 3 page fault = 9

FILE ~~CREATE~~, OPEN, WRITE(), CLOSE() & READ()

```
#include <stdio.h> <fcntl.h> <unistd.h>
108
int main(){ int fd; char buffer[20];
    fd = open("file.txt", O_CREAT|O_RDWR, 0644);
     write(fd, "Hello OS", 8);
       lseek(fd, 0, SEEK_SET);
       read(fd, buffer, 8);
       printf(" file content %s", buffer);
     close(fd);
    return 0;
}
```

READER & WRITER

```
#include <stdio.h> <stdlib.h> <pthread.h> <semaphore.h>
int count =0, rcount =0;
 sem_t mutex, wr;
void * writer (void *p){
    int *i = (int*)p;
    sem_wait(&wr);
    void * writer (void *p) {
          int *i = (int*) p;
          sem_wait(&wr)
    pf(" Writer %d writes page no %d", *i,
       ++count);
    sem_post(&wr);
  }
void * reader (void *p) {
      int *i = (int *)p;
      sem_wait (&mutex);
      rcount ++;
      if (rcount ==1)
            sem_wait (&wr);
            sem_post (&mutex);
             pf("Reader %d reads pg no %d", *i, count);
```

```
        sem_wait(&mutex);
        rcount --;
if (rcont ==0)
        sem_post(&wr);
        sem_wait(&mutex);
}
int main() {
        sem_init(&mutex, 0, 1);
        sem_init(&wr, 0, 1);
        int a[6]={1,2,3,1,2,3};
        pthread_t p[6];
        for (int i=0; i<3; i++)
                pthread_create(&p[i],NULL,writer,
                        &a[i]);
        for (int i= 3; i<6; i++)
                pthread_create(&p[i],NULL,reader,
                        &a[i]);
        for (int i=0; i<6; i++)
                pthread_join(p[i],NULL);
}
```

PRODUCER - CONSUMER :

```
void initialize() {
        pthread_mutex_init(&mutex,NULL);
        sem_init(&full,1,0);
        sem_init(&empty,1,buffersize);
        count =0;
}
void writer(int item){
        buffer[counter++] = item;  }
void reader() {
        return(buffer[--counter]);
}
```

```
void * producer (void * param) {
        int waittime, item, i;
        item = rand() % 5;
        waittime = rand() % 5;
        sem_wait (&empty);
        pthread_mutex_lock (&mutex);
        pf ("Producer has produced item: %d ", item);
        write(item);
        pthread_mutex_unlock (&mutex);
        sem_post(&full);
}
void * consumer (void* param){
        int waittime, time;
        waittime = rand() % 5;
        sem_wait (&full);
        pthread_mutex_lock (&mutex);
        item = read;
        pf ("Consumer has consumed item %d ", item);
        pthread_mutex_unlock (&mutex);
        sem_post (&empty);
}
```

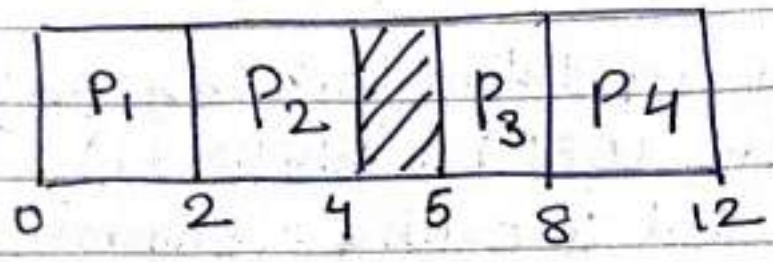
DINING:

```
# define N 5 # THINKING 2 # HUNGRY 1 # EATING 0
# LEFT (phnum+4) % N # RIGHT (phnum + 1) % N
int State[N];
int phil[N]={0,1,2,3,4};
sem_t mutex;
sem_t S[N];
Void test (phnum){
    if (State [phnum] == HUNGRY && State [LEFT]
        != HUNGRY EATING && State [RIGHT] != EATING)
```

```
{ state [phnum] = EATING;
    sleep(2);
    pf (" Philosopher %d takes fork %d & %d",
        phnum+1, LEFT+1, phnum+1);
    pf(" Philosopher %d is Eating ", phnum+1);
    sem_post (&S[phnum]);
}  }
void take-fork (int phnum){
        sem_wait (&mutex);
        state [phnum] = HUNGRY;
        pf ("Philosopher %d is hungry", phnum+1);
        test (phnum);
        sem_post (&mutex);
        sem_wait(&S[phnum]);
        sleep(1);
}
void put_fork (int phnum){
        sem_wait (&mutex);
        state [phnum]=THINKING;
        pf(" Philosopher %d putting fork %d & %d
            down", phnum+1, LEFT+1, phnum+1);
        test(LEFT);
        test (RIGHT);
        sem_post(&mutex);
}
void * philosopher(void * num) {
        while (1){
            int *i = num;
            sleep(1);
            take_fork (*i);
            sleep(0);
            put_fork (*i); } }
```

FIRST COME FIRST SERVE

Gantt Chart

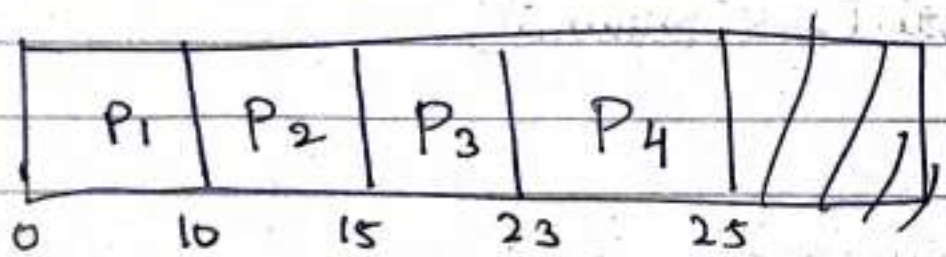


P	AT	BT
P1	0	2
P2	1	2
P3	5	3
P4	6	4

Process	TAT = CT - AT	WT = TAT - BT
P1	2 - 0 = 0	0 - 2 = -2
P2	4 - 1 = 3	3 - 2 = 1
P3	8 - 5 = 3	3 - 3 = 0
P4	12 - 6 = 6	6 - 4 = 2

ATAT: $\frac{14}{5} = 3.5$

AWT



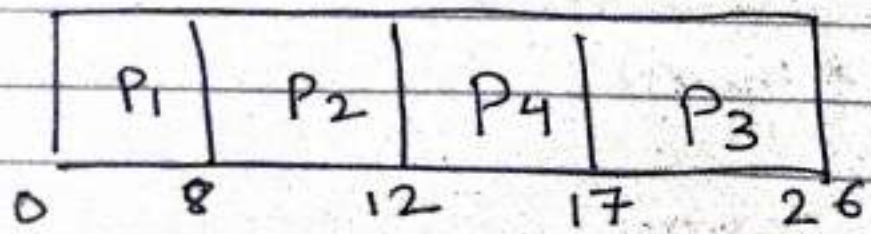
P	AT	BT
P1	0	10
P2	2	5
P3	4	8
P4	6	2

Process	TAT = CT - AT	WT = TAT - BT
P1	10 - 0 = 10	10 - 10 = 0
P2	15 - 2 = 13	13 - 5 = 8
P3	23 - 4 = 19	19 - 8 = 11
P4	25 - 6 = 19	19 - 2 = 17

ATAT: $\frac{61}{4} = 15.25$

AWT: $\frac{36}{4} = 9$

SJF - Shortest Job First



P	AT	BT
P1	0	8
P2	1	4
P3	2	9
P4	3	5

P_1	P_4	P_2	P_3

0 10 ~~12~~ 17 25

	AT	BT
P_1	4	5
P_2	6	4
3	0	3
4	6	2
5	5	4

P_3	~~P_4~~	P_1	P_4	P_5	P_2

0 3 4 9 11 15 19

P	AT	BT
1	0	8
2	1	4
3	2	9
4	3	5

P_1	P_2	P_4	P_1	P_3

0 ~~8~~ 5 10 17 26

$P_1 \rightarrow$ ~~8~~ 7 $P_2 \rightarrow$ ~~4~~ 0 $P_4 \rightarrow$ ~~5~~ 4